

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA11

COURSE OUTCOME COVERED

***CO1:** Apply object-oriented concepts and software development life cycle models to design structured and modular software systems. (BL2, BL3)*

Assessment Tool Weightage: 60%

Dr Ramamoorthy

QUESTIONS: 12

Total Marks:60

Annexure A

CONCEPT MAPPING

Code of Conduct:

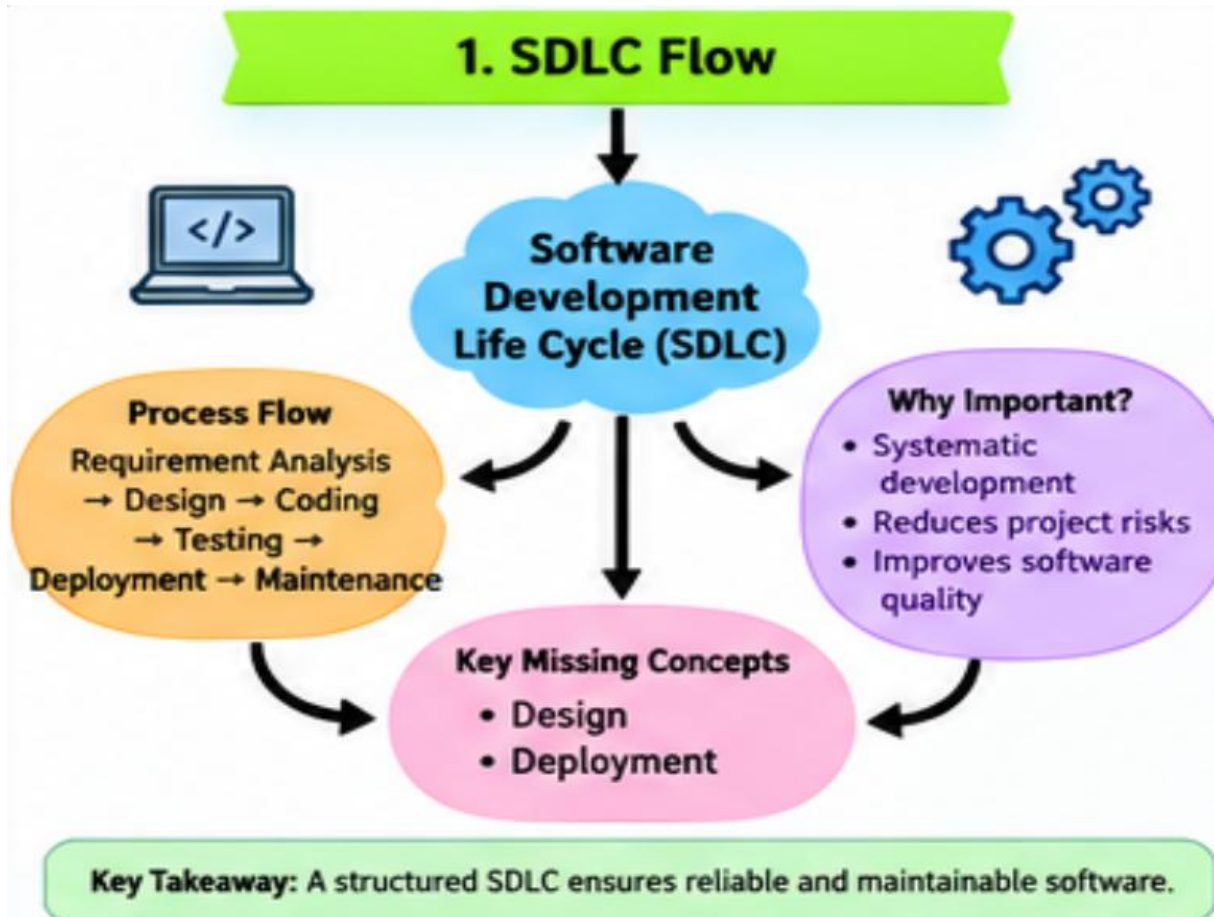
I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

CONCEPT MAPPING

Question 1 (5 Marks)



Theory: Requirement Analysis → Design → Coding → Testing → Deployment → Maintenance

Explanation

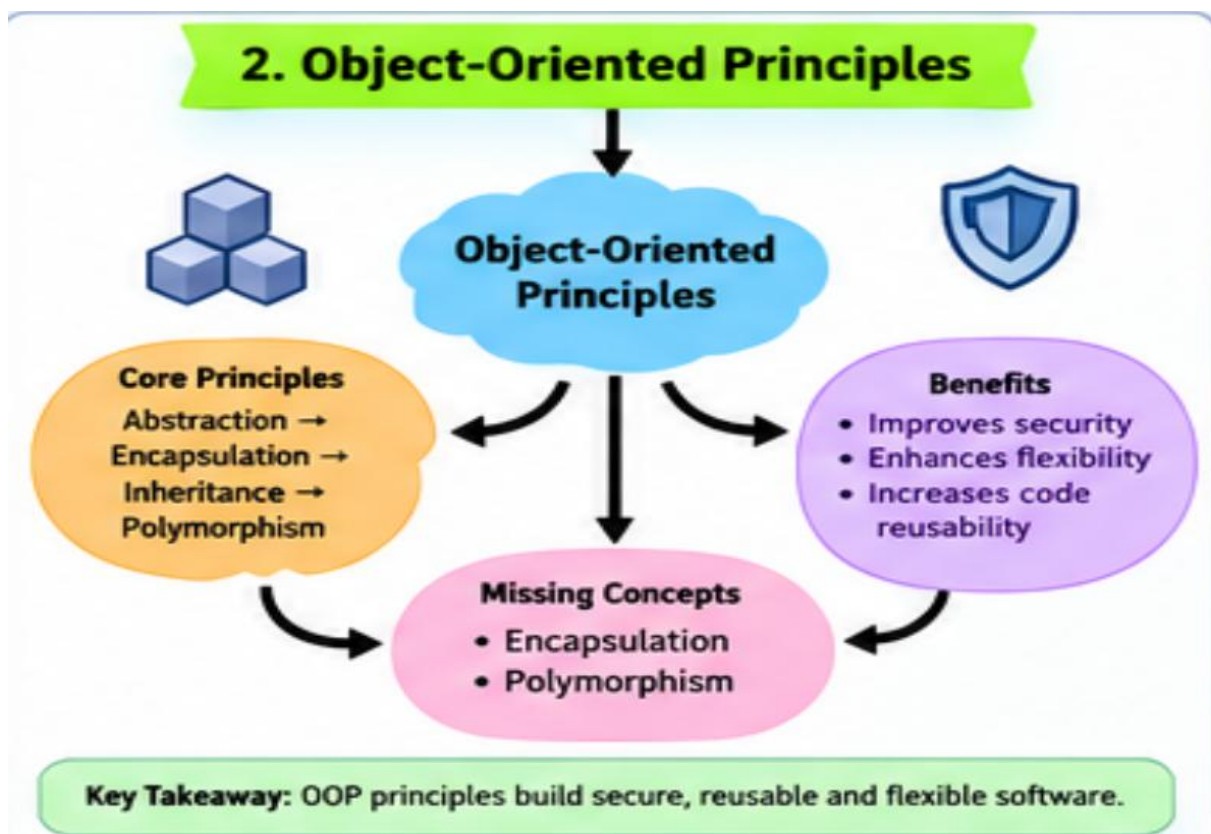
The Software Development Life Cycle (SDLC) is a structured process used to develop software efficiently. After gathering user requirements, the **Design** phase creates the system architecture, database design, and user interface. The **Coding** phase converts the design into source code. The software is then verified during **Testing** to detect and fix errors. After successful testing, the software is released through **Deployment**. Finally, **Maintenance** ensures the software continues to work properly by fixing bugs, improving performance, and adding new features when required.

Analysis

- Provides a systematic development process.
- Reduces project risks.
- Improves software quality.
- Makes maintenance easier.

Question 2 (5 Marks)

Theory: Object-Oriented Principles



Explanation

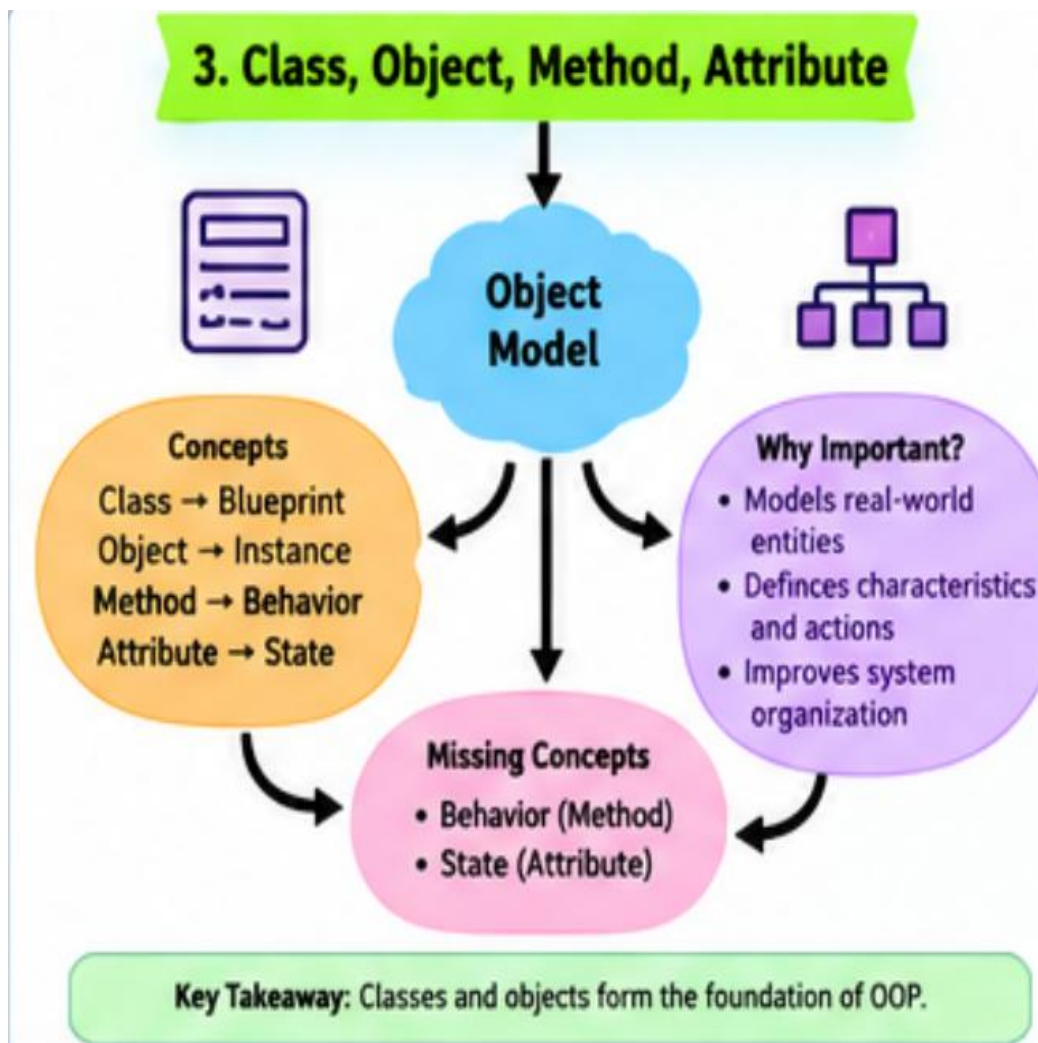
Object-Oriented Programming (OOP) is based on four important principles: **Abstraction**, **Encapsulation**, **Inheritance**, and **Polymorphism**. Abstraction hides unnecessary implementation details and displays only essential features. Encapsulation protects data by restricting direct access. Inheritance allows one class to inherit properties and methods from another class, promoting code reuse. Polymorphism enables one interface to perform different functions depending on the object. These principles make software more secure, reusable, flexible, and easier to maintain.

Analysis

- Improves software security.
- Enhances code reusability.
- Supports flexible programming.
- Simplifies maintenance.

Question 3 (5 Marks)

Theory: Class, Object, Method and Attribute



Explanation

A **Class** is a blueprint that defines the properties and behaviors of an object. An **Object** is an instance of a class that represents a real-world entity. A **Method** defines the behavior or actions performed by an object, while an **Attribute** stores the object's data or state. Together, these concepts help represent real-world systems effectively and form the foundation of object-oriented programming.

Analysis

- Models real-world entities.
- Defines object behavior and state.
- Improves software organization.
- Supports modular programming.

Question 4 (5 Marks)

Theory: Software Quality



Explanation

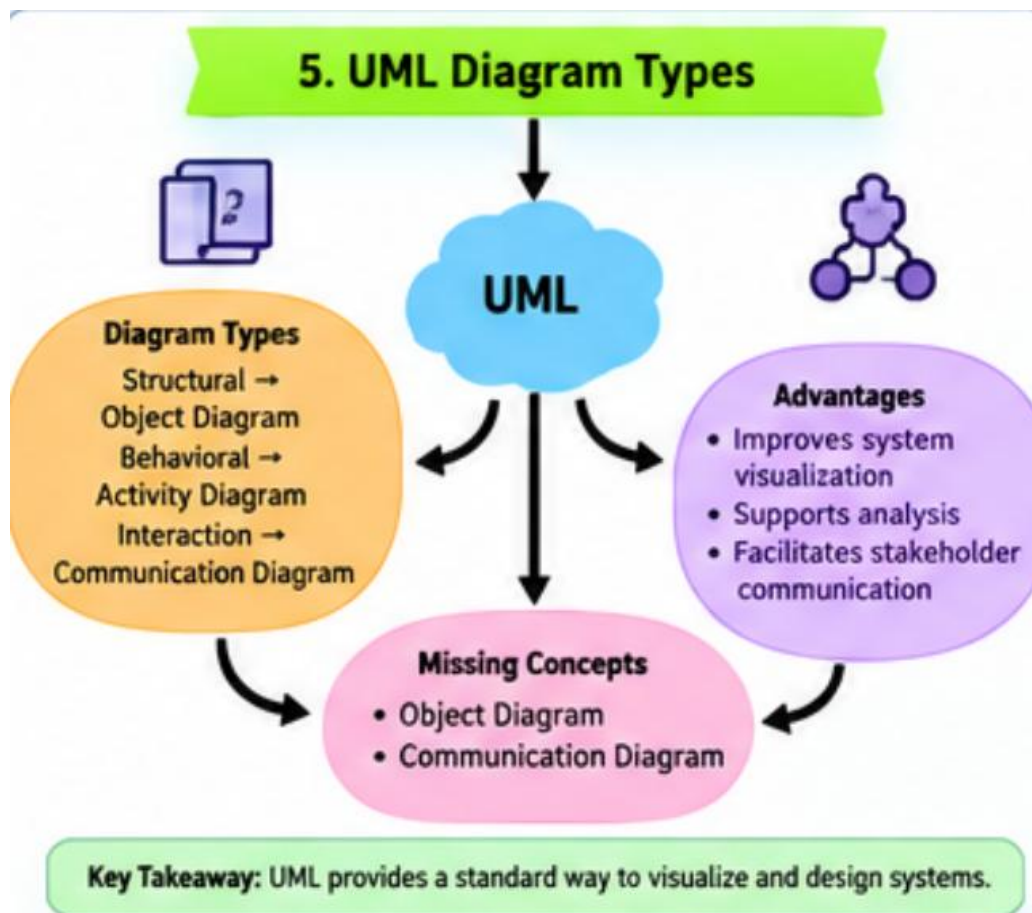
Software quality is measured using several quality factors such as **Reliability**, **Usability**, and **Maintainability**. Reliability ensures that the software performs correctly under different conditions. Usability measures how easily users can understand and operate the software. Maintainability refers to the ease of modifying, updating, and fixing the software after deployment. A software product with high quality provides better performance, customer satisfaction, and long-term usability.

Analysis

- Improves user satisfaction.
- Enhances product acceptance.
- Reduces maintenance effort.
- Increases software reliability.

Question 5 (5 Marks)

Theory: UML Diagram Types



Explanation

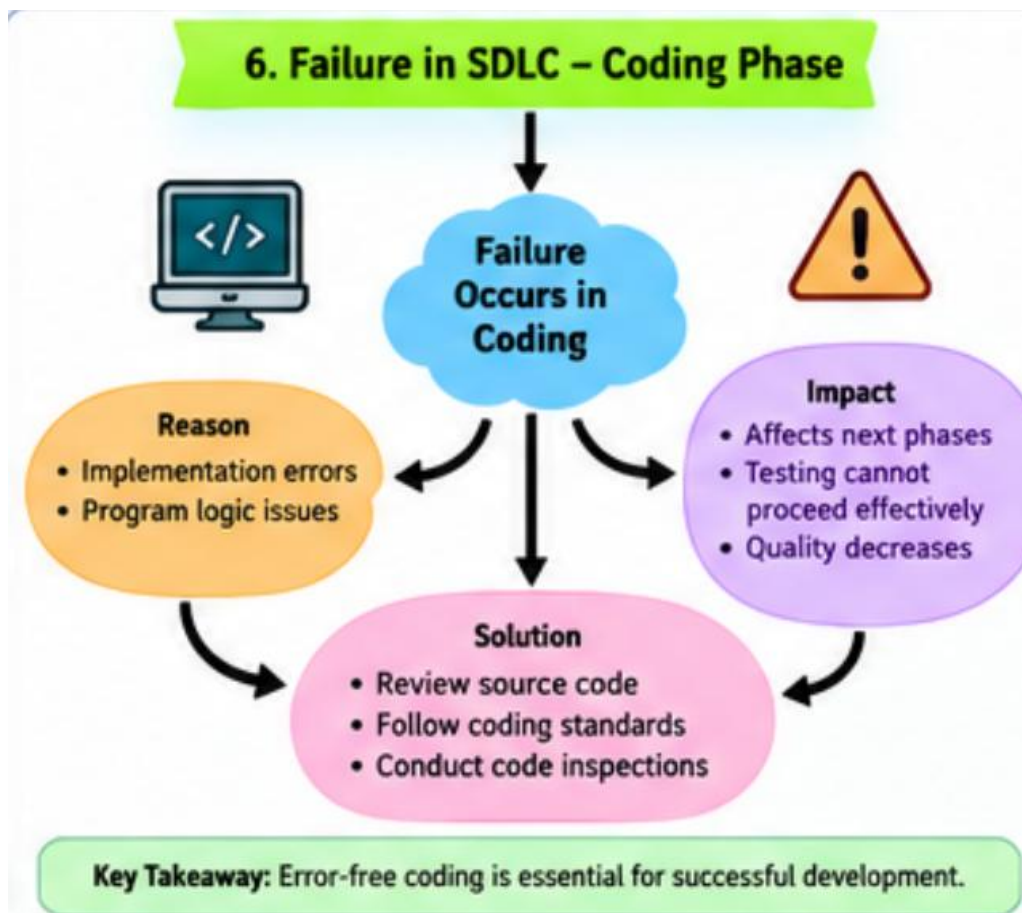
Unified Modeling Language (UML) is a standard modeling language used to visualize, design, and document software systems. UML contains different types of diagrams. **Structural Diagrams** represent the static structure of the system, including the **Object Diagram**. **Behavioral Diagrams** describe system behavior, including the **Activity Diagram**. **Interaction Diagrams**, such as the **Communication Diagram**, show how objects exchange messages. UML diagrams help developers understand and communicate software design effectively.

Analysis

- Improves system visualization.
 - Supports software analysis.
 - Enhances documentation.
 - Facilitates stakeholder communication.
-

Question 6 (5 Marks)

Theory: Failure in Coding Phase



Explanation

The Coding phase is responsible for converting system design into a working program. If implementation errors occur during coding, they affect all later phases of software development. Logical mistakes, syntax errors, or incorrect algorithms may prevent testing from proceeding successfully. These problems reduce software quality and increase debugging time. Following coding standards and reviewing source code regularly helps minimize errors during implementation.

Analysis

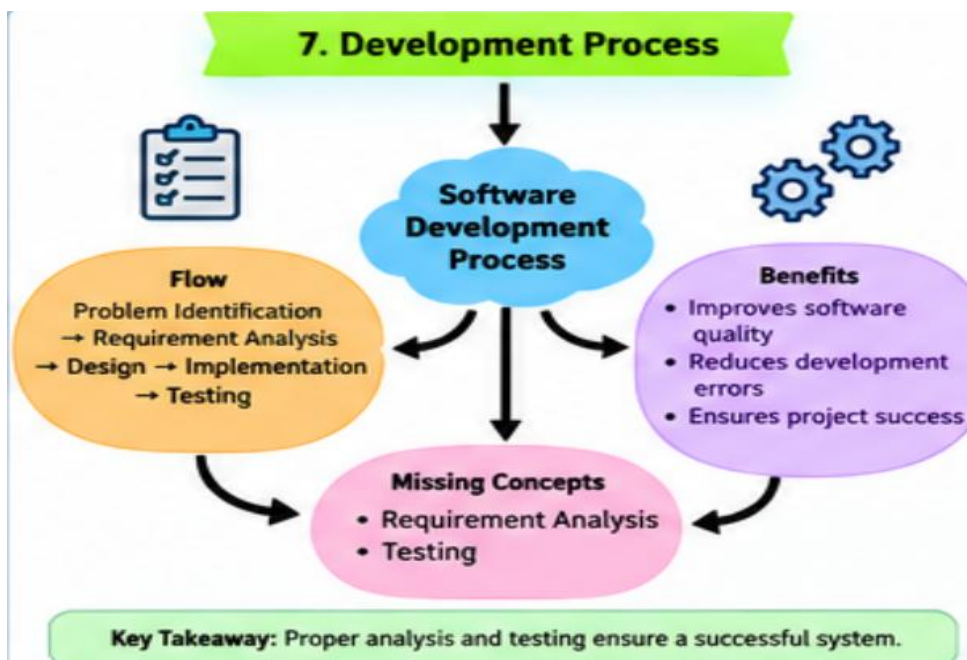
- Errors affect later phases.
- Testing becomes difficult.
- Software quality decreases.
- Debugging effort increases.

Solution

- Review the source code.
- Follow coding standards.
- Conduct code inspections.
- Perform unit testing before integration.

Question 7 (5 Marks)

Theory: Problem Identification → Requirement Analysis → Design → Implementation → Testing



Explanation

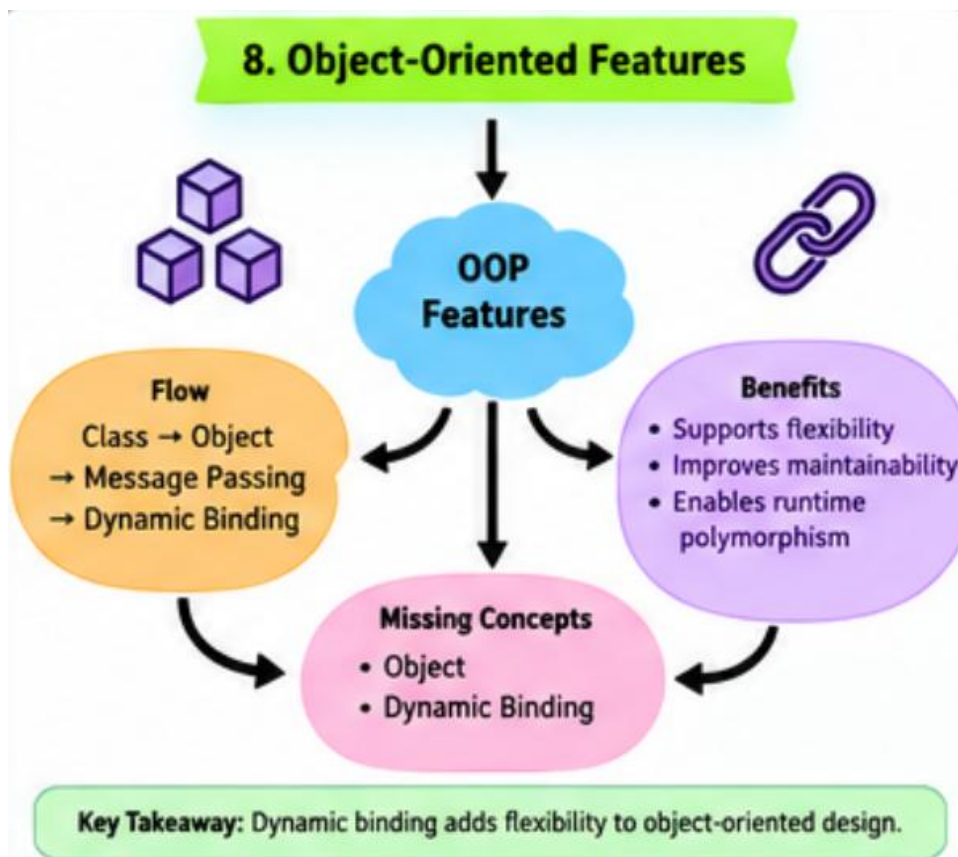
Software development begins with **Problem Identification**, where the existing issue or user need is recognized. The next step is **Requirement Analysis**, which gathers and documents the functional and non-functional requirements of the system. During the **Design** phase, the software architecture, database, and user interface are planned. In the **Implementation** phase, developers write the source code based on the design. Finally, **Testing** verifies that the software functions correctly and satisfies all user requirements before deployment.

Analysis

- Improves software quality.
- Reduces development errors.
- Ensures customer requirements are met.
- Increases project success.

Question 8 (5 Marks)

Theory: Object-Oriented Features



Explanation

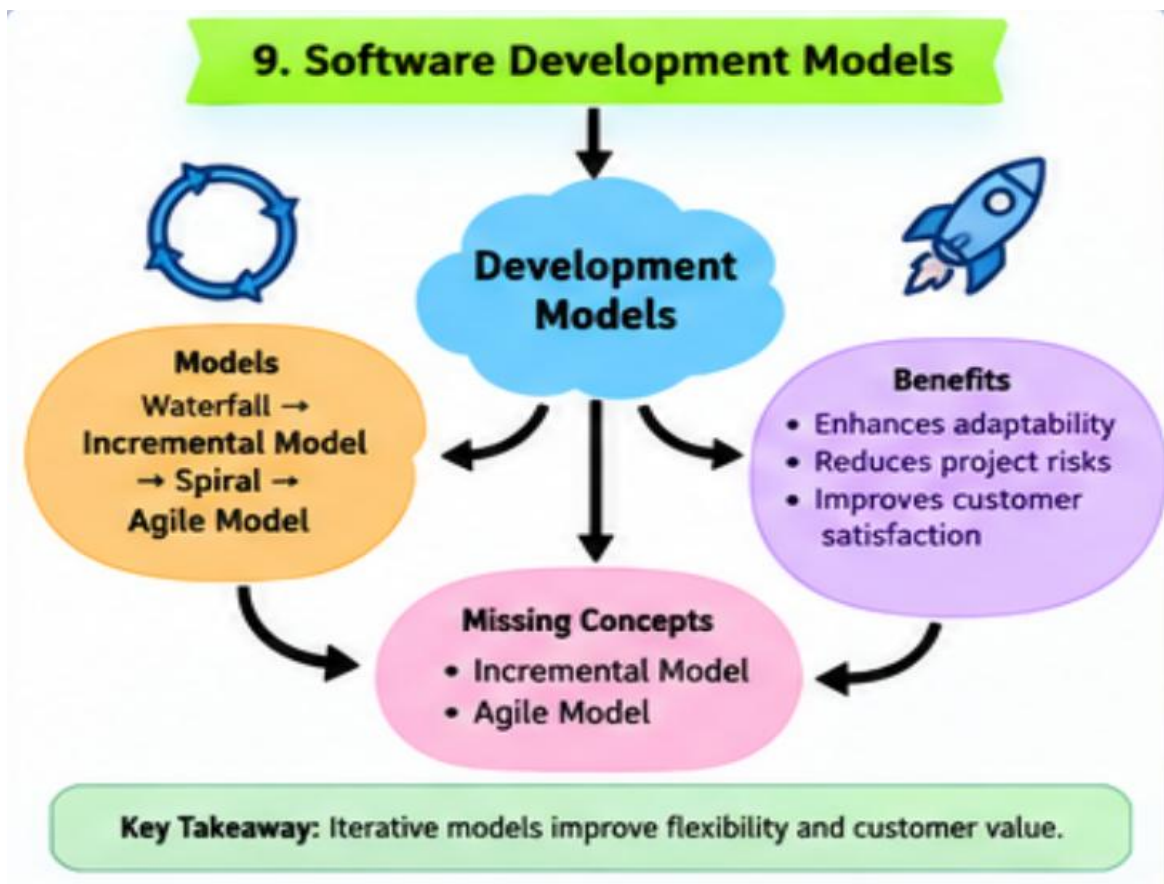
Object-Oriented Programming (OOP) is based on important features such as **Class**, **Object**, **Message Passing**, and **Dynamic Binding**. A **Class** acts as a blueprint for creating objects. An **Object** is an instance of a class that contains data and methods. **Message Passing** allows objects to communicate by calling methods of other objects. **Dynamic Binding** determines which method should execute during runtime, making the software more flexible and supporting runtime polymorphism. These features improve modularity, flexibility, and software reuse.

Analysis

- Supports flexible software design.
- Improves code maintainability.
- Enables runtime polymorphism.
- Enhances software reusability.

Question 9 (5 Marks)

Theory: Software Development Models



Explanation

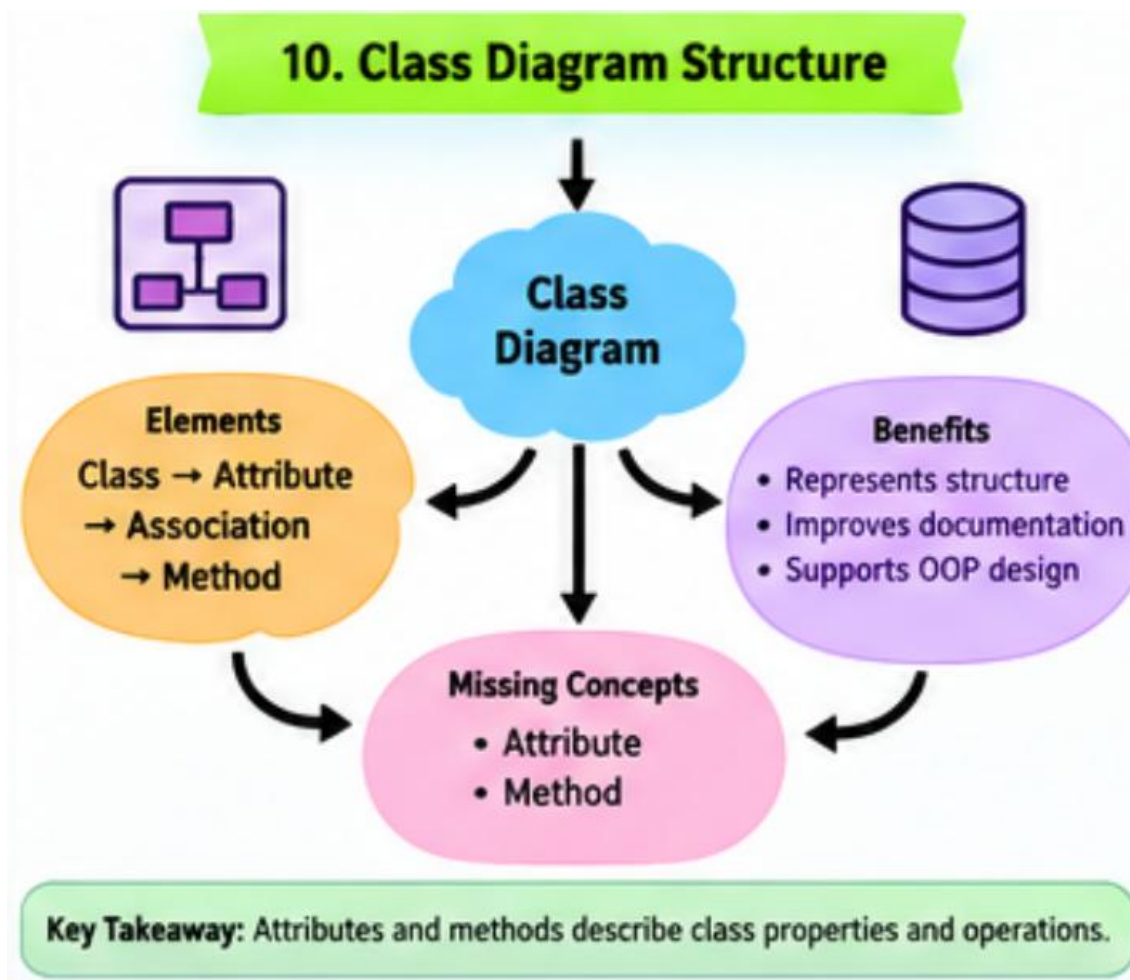
Software Development Models define the approach used to develop software projects. The **Waterfall Model** follows a sequential process where each phase is completed before the next begins. The **Incremental Model** develops software in smaller modules or increments, allowing partial delivery of the system. The **Spiral Model** combines development with continuous risk analysis and customer feedback. The **Agile Model** supports iterative development, continuous improvement, and close customer involvement. Choosing the appropriate model depends on project size, complexity, and customer requirements.

Analysis

- Enhances project adaptability.
- Reduces development risks.
- Supports faster software delivery.
- Improves customer satisfaction.

Question 10 (5 Marks)

Theory: Class Diagram



Explanation

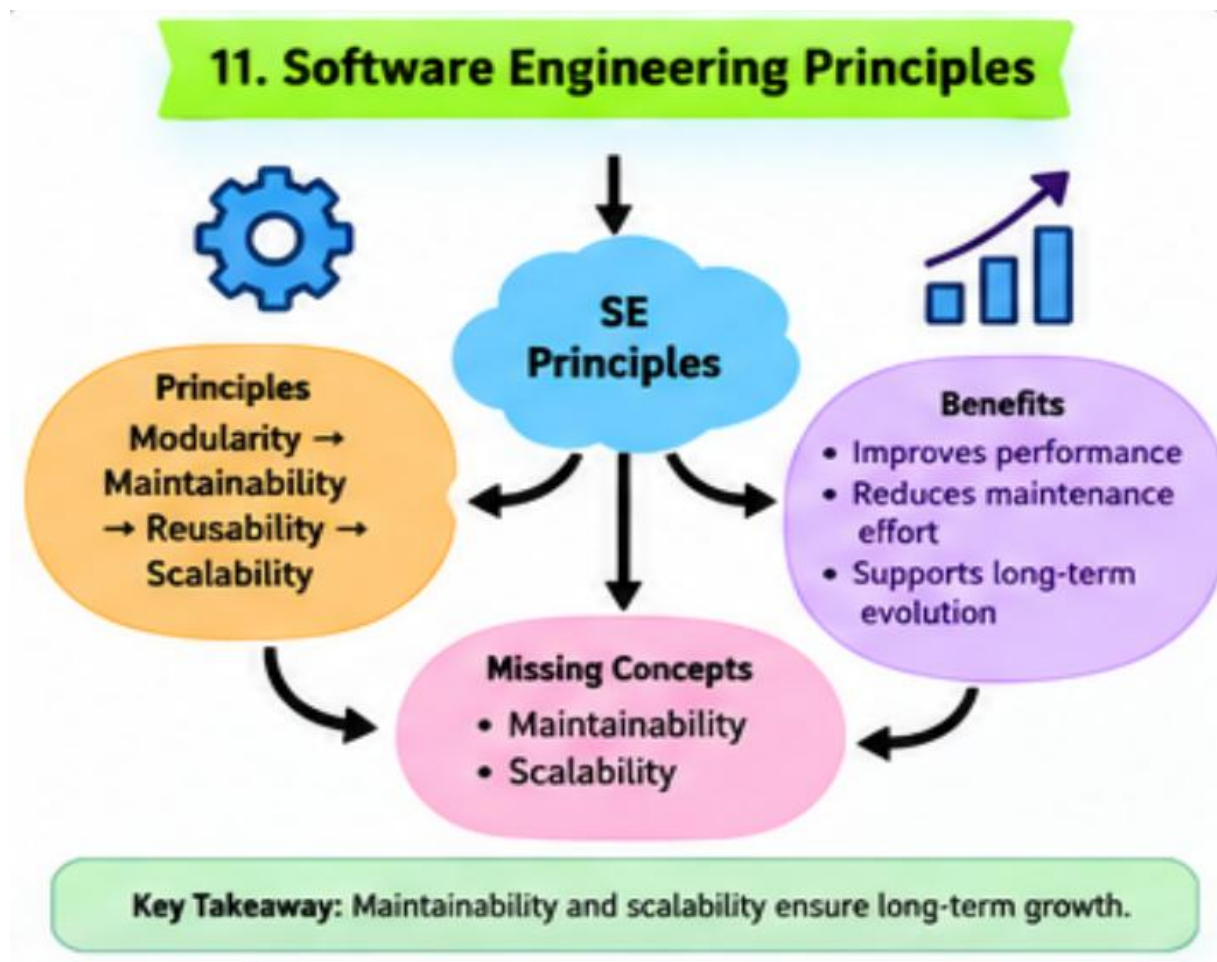
A **Class Diagram** is one of the most important UML structural diagrams used to represent the static structure of an object-oriented system. It shows the classes, their **Attributes**, **Methods**, and the relationships between classes such as Association, Aggregation, Composition, and Inheritance. Attributes represent the properties or data members of a class, while methods define the operations or functions performed by the class. Class diagrams help developers understand system structure before implementation.

Analysis

- Represents software structure clearly.
- Improves software documentation.
- Supports object-oriented design.
- Makes system development easier.

Question 11 (5 Marks)

Theory: Software Engineering Principles



Explanation

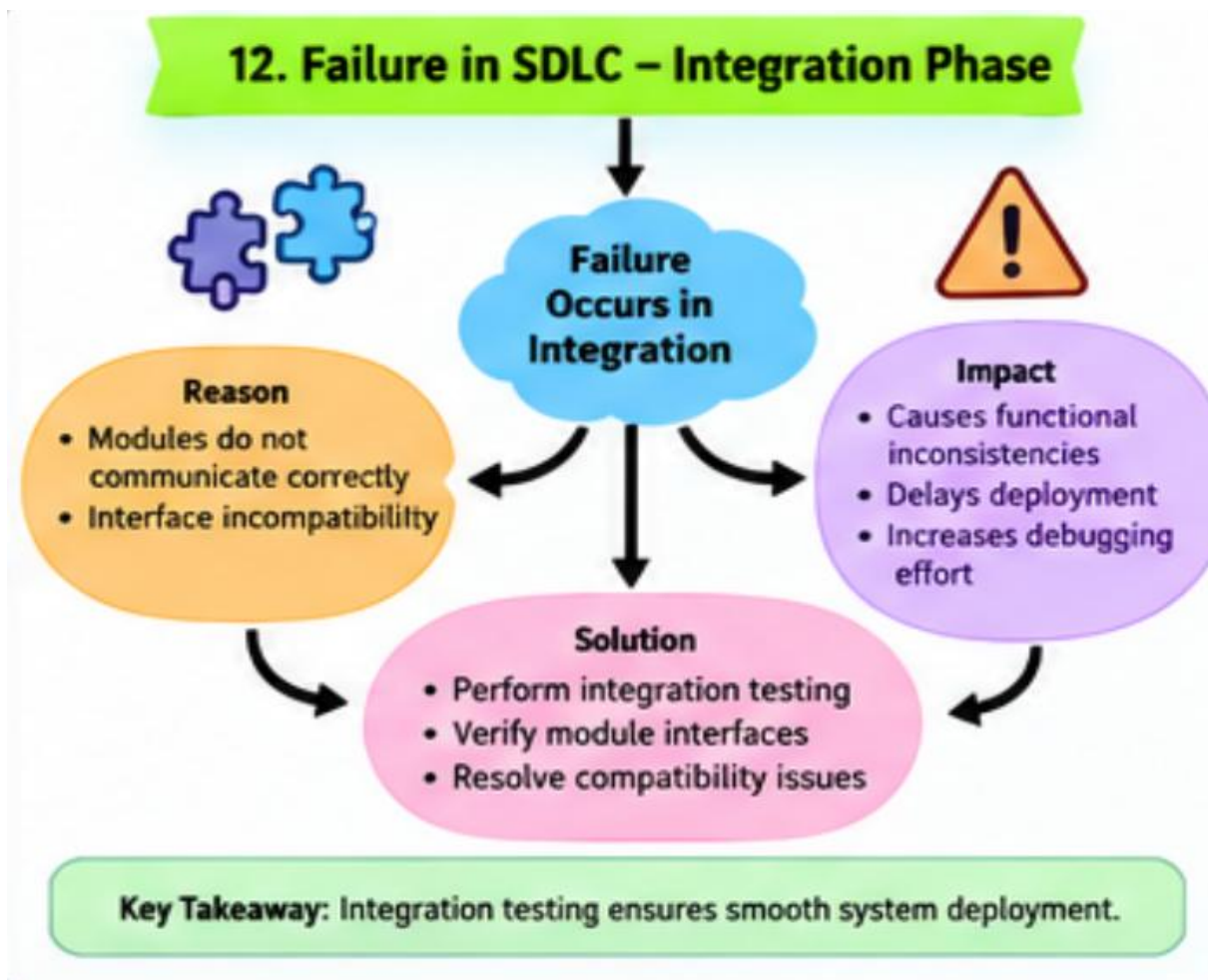
Software Engineering Principles provide guidelines for developing reliable and maintainable software systems. **Modularity** divides software into independent modules, making development and testing easier. **Maintainability** allows software to be modified, updated, and repaired efficiently. **Reusability** enables existing software components to be used in multiple projects, reducing development time. **Scalability** allows the software to handle increasing users, data, and workload without reducing performance. These principles improve software quality and support long-term software evolution.

Analysis

- Improves software performance.
- Reduces maintenance effort.
- Supports software scalability.
- Encourages component reuse.

Question 12 (5 Marks)

Theory: Failure in Integration Phase



Explanation

The **Integration Phase** combines individual software modules into a complete system. If modules fail to communicate correctly, integration errors occur. Common reasons include incompatible interfaces, incorrect data exchange, and communication failures between modules. These issues lead to functional inconsistencies, increase debugging effort, and delay software deployment. Proper integration testing helps identify these problems before the software is released.

Analysis

- Causes communication failures between modules.
- Delays software deployment.
- Increases debugging effort.
- Reduces overall software quality.

Solution

- Perform integration testing.
- Verify module interfaces.
- Resolve compatibility issues.
- Re-test the integrated system before deployment.

CONCEPT MAPPING

RUBRICS FOR CALCULATION

CONCEPT MAPPING

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand